

ML Summer Bridge: Week 1 Study Notes

Machine Learning Class

Summer 2026

Week 1: Analyzing Data

Remember the table that we introduced at the end of last week:

Father's Height	Mother's Height	Gender	Height
70.2	60.5	M	72.545
68.4	62.2	F	69.03
73.7	64.1	M	76.265
72.5	64.4	F	72.48
74.6	68.2	F	75.45

This table has 4 columns. If we include the column headers then the table is said to have 6 rows. However, we generally do *not* include the column headers in the row count. Therefore this table has 5 rows. We can talk about the *shape* of the table, which is an ordered pair (numRows, numCols). This table has a shape equal to (5, 4). Each row in the table represents one entry, sometimes called a *data point*. In this table, each data point has four pertinent pieces of information: the height of an individual's father, the height of their mother, their gender and their own height. The first data point in the table corresponds to the first row, and can be represented as the list [70.2, 60.5, M, 72.545]. What would be the list corresponding to the second data point?

In order to get this table into Google Colab, make a new cell by hitting "+ Code" towards the top-left of the Window. In this cell paste the following:

```
url = "https://docs.google.com/spreadsheets/d/1rNf2GZB5_a3KAh_AR-Tb_xfU2Z9wFENkEYmVvEc08Gg/export?format=csv&gid=0"
data = pd.read_csv(url)
print(data)
```

If you get an error, there are two things that you should make sure you have: first, while the URL here in split up onto two lines, make sure that the URL is a single, long string rather than split up over two lines. Second, as mentioned in the Week 0 document, at the top of every Google Colab file, you should write in the first cell the following lines:

```
import pandas as pd
import numpy as np
```

In the above we let 'url' store the string which contains the url of a Google Sheet I created which stores the same data as our table. We then used pandas read_csv() method to read in the table and store it as a type of object called a pandas DataFrame. Our DataFrame is named 'data.' The final line printed out our data.

There are a few commands that will be very helpful for us in accessing information from our DataFrame named 'data.' The first few of these commands allow us to access certain rows, columns and specific elements of our DataFrame.

To access a specific column, say the 'Gender' column we can use the command:

```
print(data[ 'Gender ' ])
```

We can access the 'Mother's Height' column similarly:

```
print(data[ 'Mother 's  Height ' ])
```

If we want to access a row, a column or a specific element based on its numerical index within 'data' we can use the command `iloc`. Before we get into how `iloc` works, we will mention two important facts: the first is that in computer science, we start counting at 0. In other words, if we have five rows, we label the rows 0, 1, 2, 3, 4 and not 1, 2, 3, 4, 5. The second is that to access an element we first specify the row and then the column. Lets see an example:

```
print(data.iloc[0 , 3])
```

This should print the number 72.545 because we are looking at the row labeled zero (the first row) and the column labeled 3 (the fourth column). The element here is 72.545. Take a moment to see if you can figure out what you would get if you run the following:

```
print(data.iloc[0 , 2])
print(data.iloc[2 , 0])
print(data.iloc[3 , 0])
print(data.iloc[3 , 1])
print(data.iloc[1 , 3])
print(data.iloc[4 , 2])
```

Were you correct?

We can also use `iloc` to change the value of an entry in our table. Let's say we realized we made a mistake and the second person's mother is actually 63.2 inches tall, not 62.2. We can adjust her height in the following manner:

```
data.iloc[1 , 1] = 63.2
```

We can also use `iloc` to get an entire row or column. To access the first row we write:

```
print(data.iloc[0 , :])
```

The 0 inside `iloc` indicates we are interested in the row with index 0 (the first row). The colon instead of a

second number means "give us all the entries in that row." To access the second *column* we could similarly write:

```
print(data.iloc[:, 1])
```

Here, the 1 in the second position indicates we are interested in the column at index 1 (the second column). The semicolon where the row number would be located indicates that we want all the entries in that column. How would you get the third column? What about the third row? Finally, if we want to access an element by referring to the row number and the column's *name* we can do that in the following manner:

```
print(data.at[1, 'Gender'])
```

This returns the element at row index 1, and column 'Gender.' What element is this?

Notice how for most of the columns the data is a number. However, for the gender column the data is a letter: 'M' or 'F' indicating whether the person is male or female. While this conveys the gender information succinctly and compactly, when we would do a numerical analysis on the data, it becomes difficult when some of the data isn't numerical. We need a way to convert the gender information to numerical information. How might we do that?

We will address this issue by associating a number with 'M' and a number with 'F.' The best way to do this (and we will explain why any other numbering scheme causes issues when we return in the fall) is to assign a '1' to one of the genders, say - male. We will assign a '0' to the other gender - female. We could equally well have assigned female to '1' and male to '0.' It is not important which class gets '1' and which gets '0' - just that we are consistent and that we use the numbers '1' and '0' and not some other numbering system like '1' and '2.' **If you can figure out why we shouldn't use a different numbering scheme, let me know during our first class and you will receive extra credit on the first exam.**

We will modify our dataframe to replace all occurrences of the letter 'M' in the gender column with a 1, and all occurrences of the letter 'F' with a 0. The code below accomplishes that:

```
for i in range(data.shape[0]): #shape[0] is the number of rows
    if(data.at[i, 'Gender'] == 'M'):
        data.at[i, 'Gender'] = 1
    else:
        data.at[i, 'Gender'] = 0
print(data)
```

Can you explain what each line of the above code does? Great. There are two final commands we will learn here. The first command 'shape' we actually used in the code above. It returns a 'tuple' of the form (numRows, numCols). For example:

```
print(data.shape)
```

This prints (5,4) as data has 5 rows and 4 columns. Finally we will mention that we can use a command 'loc' (which is distinct from *iloc*) to add or change a row:

```
data.loc[3] = [75.6, 69.2, 1, 77.2]
```

This changes what was at the row with index 3 to the list where the father's height is 78, the mother's height is 65, their gender is 1 (male) and they are 73 inches tall. Print the dataframe to see the changes. Finally, if we have rows labelled

$$[1, \dots, n]$$

we can use 'loc' with input $n + 1$ to add a row:

```
data.loc[5] = [66.8, 65.4, 1, 72.18]
```

Since our dataframe 'data' had five rows with indices [0,...,4] we use .loc[5] to add a row where the father's height is 62 inches, the mother's height is 64 inches, their gender is 1 (male) and they are 61 inches tall. After running this command print the shape of data. Did it change? Now that we have 6 rows (what are their labels) what should we write to add another row to the bottom of our table (you can choose what data the father's height, mother's height, gender and height should be).

Great job on getting to this point! Take a break, make sure you have absorbed all the information and are well-rested. When you are ready, move on to week 2's material.